talenciaglobal

RADIX

Talent Match Hackathon

Solution Document Template

Submission format: one Google Drive folder link containing this document (PDF), the demo video, and the code.

Team Name	Piratech
Team Members & Roles	• Aman Sharma – JD Analytics – sharma.aman67284@gmail.com • Aryan Nigam – Resume Parsing [Scrum Master] – nigamaryan18110@gmail.com • Khushi Goel – Profile Builder – kg5078856@gmail.com • Vaibhav Maurya – Talent Check – vaibhavvns2023@gmail.com • Keshav Agrawal – Skill Matching – keshavagr.ka07@gmail.com
Google Drive Folder Link	https://drive.google.com/drive/folders/1Piratech_RADIX_TalentMatch_2026?usp=sharing [Viewer Access]
Demo Video (15 min)	Demo_Video.mp4 in the Drive folder
Date of Submission	28/10/2026 | Deadline: Wednesday, 28 October 2026, 4:00 PM IST

Table of Contents

- How to Submit
- How Your Solution Will Be Evaluated
- Submission Details
- 1. Problem & Solution Summary
- 2. What Makes Our Solution Different
- 3. Team & Roles
- 4. Shared Data Contract
- 5. How Each Piece Works
- 6. Integrated Flow & Tech Stack
- 7. Quality Evidence
- 8. Results on Sample Data
- 9. How to Run Our Code
- 10. Definition of Done
- 11. Challenge, Learnings, Limitations & Next Steps
- 12. Demo Video (15 Minutes)
- 13. Declarations

How to Submit

Each team submits one Google Drive folder link containing this document (as PDF), a 15-minute demo video and the code, by Wednesday, 28 October 2026, 4:00 PM IST. Evidence beats description: wherever you can show a number, a screenshot or a JSON sample, do that instead of explaining.

1. Create one folder in Google Drive named TeamName_RADIX_TalentMatch (Piratech_RADIX_TalentMatch).
2. Upload into it: Solution_Document.pdf, Demo_Video.mp4, Code.zip (with README.md, without API keys).
3. Share folder: General access -> Anyone with the link -> Viewer.
4. Tested in incognito mode to guarantee seamless public access.

How Your Solution Will Be Evaluated

Area	What Evaluators Look For	Evidence in	Weight
Innovation & value	A clear candidate problem, a specific one-line difference, and features beyond the brief explained and visible in demo.	Sec. 1, 2	**20%**
Build & architecture	All five pieces working as one automated flow, a complete and validated data contract, and well-reasoned design decisions.	Sec. 3, 4, 5, 6, 10	**34%**
Quality & results	Measured extraction accuracy, honest robustness testing, and results that differ across candidates and match expected fit.	Sec. 7, 8	**32%**
Reproducibility & presentation	Code that runs from the README, a specific retrospective with honest limitations, and a live demo with every member presenting.	Sec. 9, 11, 12	**14%**

Submission Details

Role	Member Name	Email
1. JD Analytics	Aman Sharma	sharma.aman67284@gmail.com
2. Resume Parsing	Aryan Nigam	nigamaryan18110@gmail.com
3. Profile Builder	Khushi Goel	kg5078856@gmail.com
4. Talent Check	Vaibhav Maurya	vaibhavvns2023@gmail.com
5. Skill Matching	Keshav Agrawal	keshavagr.ka07@gmail.com
Scrum Master (one of the five)	Aryan Nigam	nigamaryan18110@gmail.com

1. Problem & Solution Summary

• What problem does a candidate face when deciding whether they are ready for a specific job?

Candidates face an 'opacity wall': JDs list exhaustive, unrealistic wish-lists without stating baseline thresholds, while automated ATS tools reject applications on exact string matches without feedback. Candidates cannot tell if their competencies meet elite corporate benchmarks (Google, Microsoft, Oracle) or what specific technical skills they lack for a given role.

• What does your tool do, in one or two sentences?

RADIX Talent Match provides candidates with a unified, dual-lens readiness evaluation: benchmark scoring against Tier-1 enterprise expectations across 12 standardized skill categories, combined with posting-specific semantic skill-gap matching that delivers actionable upskilling roadmaps.

• Our one-line difference:

Unlike a simple keyword matcher, our tool **evaluates deep semantic competency across 12 standardized RADIX categories, provides verifiable quote evidence for every skill, and isolates exact proficiency gaps with targeted study guidance rather than binary keyword rejections.**

• What is the one thing your team is most proud of in the build?

We are proudest of our contract-driven, ultra-resilient microservice integration: 5 distinct FastAPI backends wired to a high-speed React dashboard, supported by dual-tier persistence (live Supabase cloud + zero-downtime local JSON cache) that executes end-to-end evaluations in 1.4 seconds with 100% test pass rate.

2. What Makes Our Solution Different

#	Feature / Innovation	How It Works (1–2 lines)	Why It Helps the Candidate & Where to See It
1	**Dual-Lens Evaluation Engine**	Separates company-wide benchmark readiness (Talent Check) from specific job posting alignment (Skill Matching).	Reveals if candidate is enterprise-grade AND if they fit this specific posting. Seen: Final Results View / Video 11:30
2	**Category-Aware Semantic & Partial Matching**	Combines fuzzy string matching with RADIX category alignment (e.g. Docker matching Cloud Systems) giving 50% partial credit.	Prevents unfair disqualification due to phrasing differences between resume and JD. Seen: Skill Matching Tab / Video 12:15
3	**Actionable Missing-Skill Learning Pathways**	Calculates exact numerical gap delta between candidate level and role expectation, generating targeted study topics.	Turns rejections into immediate, structured upskilling action plans. Seen: Gap Breakdown Panel / Video 10:15
4	**Dual-Tier Resilient Persistence**	Persists profiles to Supabase cloud while maintaining an automatic local JSON cache mirror.	Guarantees 100% platform uptime and zero data loss even during network drops. Seen: Profile Builder / Video 7:30

Our Standout Feature: A dual-lens evaluation architecture that simultaneously measures baseline corporate readiness (Talent Check) and posting-specific skill fit (Skill Matching) with verifiable textual evidence and targeted study plans.

3. Team & Roles

Role	What I Built (one line)	Status
1. JD Analytics (Aman Sharma)	FastAPI service on Port 8001 extracting skills into 12 RADIX categories via Groq LLaMA-3	**Done**
2. Resume Parsing (Aryan Nigam [SM])	FastAPI service on Port 8002 parsing PDF/DOCX resumes, projects, and quote evidence	**Done**
3. Profile Builder (Khushi Goel)	Profile service on Port 8003 with 1-10 proficiency sliders, Supabase + local JSON cache	**Done**
4. Talent Check (Vaibhav Maurya)	Benchmark scoring engine on Port 8004 evaluating profiles against Tier-1 expectations	**Done**
5. Skill Matching (Keshav Agrawal)	Fuzzy semantic engine on Port 8005 comparing candidates to specific JDs with gap lists	**Done**

Scrum Master Note

How the team kept on schedule and cleared blockers (Aryan Nigam): Our team operated through twice-daily standups (10:00 AM & 8:00 PM IST) and established strict Pydantic contract definitions on Day 1. By deploying mock API servers for Resume Parsing and Profile Builder early, Roles 4 and 5 developed scoring engines concurrently without blocking dependencies, resulting in 100% milestone completion and zero integration bottlenecks.

4. Shared Data Contract

```
// Final Agreed JSON Schema Contract Models:
1. Skill Object: { skill_name: str, category_code: 'DSA'|'COD'|...|'OTHER', evidence: str, confidence: 'high'|'med'|'low', proficiency_level: 1-10 }
2. Candidate Profile: { id: str, name: str, email: str, education: str, skills: [<skill>], hackathons: [], internships: [], cv_file: str }
3. Talent Check Result: { company: str, readiness_score: float (0-100), is_ready: bool, skillset_gap: [{ category_code: str, required_level: float, candidate_level: float, gap: bool, gap_amount: float }] }
4. Skill Matching Result: { jd_source_file: str, role: str, match_score: float (0-100), matched_skills: [], partial_skills: [], missing_skills: [] }
```

Changes from the Brief's Proposed Schema

Field / Object Changed	What We Changed	Why
proficiency_level in <skill>	Added 1–10 integer scale for candidate skill rating	Enables granular deficit scoring in Talent Check rather than binary present/absent.
partial_skills in Match Result	Added array tracking category-adjacent overlaps	Grants partial credit for transferable skills (e.g. Docker for Cloud) preventing false rejections.
is_ready boolean in Talent Check	Added explicit readiness flag based on >= 70 score	Provides candidates with an unambiguous, immediate readiness decision.

How We Enforced the Contract: We enforced contracts using Pydantic v2 BaseModels with strict Enum validation on `category_code`. Any payload missing required fields triggers an immediate HTTP 422 Unprocessable Entity error. Our automated test suite `tests/integration/test_end_to_end.py` verified all 12 API boundaries with zero serialization errors.

5. How Each Piece Works

Role	Approach (LLM vs Plain Code & Why)	Sample Output & Details
1. JD Analytics (Aman Sharma)	LLM Pipeline (Groq LLaMA-3.3-70B) with rule fallback. Natural prose requires semantic reasoning to isolate requirements and map onto 12 categories.	ExtractedSkillList JSON: Extracted 10 skills with quote evidence, confidence levels, and normalized RADIX category codes.
2. Resume Parsing (Aryan Nigam)	Hybrid LLM + PyMuPDF/python-docx text parsing. Extracts complex two-column layouts into structured skills, projects, and education records.	ParsedResumeOutput JSON: Clean candidate object with technical skills mapped to categories and verified text excerpts.
3. Profile Builder (Khushi Goel)	Plain Code (FastAPI + Supabase + Local Cache). Standard CRUD logic with form validation. Deterministic speed and offline persistence are paramount.	CandidateProfile JSON: Persisted profile with unique UUID, editable 1-10 proficiency sliders, and dual cloud/cache sync.
4. Talent Check (Vaibhav Maurya)	Plain Code (Mathematical Scoring Engine). Evaluates profile against company_skillsets.json across 12 dimensions. Fast, zero-cost, deterministic.	TalentCheckResponse JSON: readiness_score (73.68/100), is_ready: true, and radar skillset_gap deltas.
5. Skill Matching (Keshav Agrawal)	Hybrid Fuzzy String Token Matching + Category Fallback. Compares candidate skills against JD requirements with 50% partial category credit.	SkillMatchingResult JSON: match_score (82.0%), matched_skills, partial_skills, and actionable missing-skill gap list.

Key Design Decisions

Decision	Options Considered	What We Chose and Why
How 'Ready' is Defined (Talent Check)	1. All-or-nothing check 2. Unweighted mean 3. Weighted index (>=70% threshold)	Chose Weighted Index (>=70%): Core categories (COD, DSA, SWE) carry higher weight. Prevents soft skill gaps from masking pure technical readiness.
How Skills are Matched (Skill Matching)	1. Exact string matching 2. Vector embeddings 3. Fuzzy token + category alignment	Chose Fuzzy + Category Alignment: Token matching handles plurals/typos; shared RADIX category grants 50% partial credit. Zero API cost and instant response.

6. Integrated Flow & Tech Stack

Integrated 5-Stage Automated Sequence: Pick a JD -> Parse Resume -> Build/Update Profile -> Run Talent Check -> Run Skill Matching

Integrated Tech Stack Architecture

Layer	What We Used
Language & Backend	Python 3.10+ (tested on Python 3.13) with 5 FastAPI microservices & Uvicorn
Frontend	React 18 + TypeScript + Vite + Tailwind CSS + Lucide Icons
Storage	Supabase PostgreSQL Cloud Database + Resilient Local JSON Storage Mirror
LLM Model(s)	Groq LLaMA-3.3-70B-versatile / LLaMA-3.1-8B-instant with regex/NLP heuristics
Libraries & Parsers	Pydantic v2, PyMuPDF (fitz), python-docx, pypdf, requests, python-dotenv
Speed & Cost	Full flow execution: ~1.4 seconds; 2 LLM calls per run (~$0.0003 or Free Tier)

7. Quality Evidence

7a. Extraction Accuracy (JD Analytics & Resume Parsing)

Document	Ground-Truth Skills	Extracted by Tool	Correct	Precision	Recall
JD: Google LLC — SWE	10	11	10	90.9%	100.0%
JD: Microsoft — Software Engineer	9	9	9	100.0%	100.0%
Resume: Rohan Verma	12	13	12	92.3%	100.0%
Resume: Ananya Rao	11	11	11	100.0%	100.0%

In 2 lines: what did the tool miss or get wrong, and what did you change because of it?
Initial extraction over-captured broad prose concepts like 'cross-functional collaboration' under SWE. We engineered strict negative boundary prompts and an explicit non-technical stoplist, ensuring soft-skills route strictly to COMM or OTHER while preserving 100% technical recall.

7b. Robustness Tests

Scenario	What Happened	Pass / Partial / Fail
Same resume in PDF and DOCX — same skills extracted?	PyMuPDF and python-docx extracted identical skills with 100% attribute parity.	**Pass**
Same JD run twice — stable output?	Deterministic prompt (temperature=0.0) produced identical skill lists and category tags.	**Pass**
A messy or two-column resume layout	PyMuPDF reading-order sorting prevented horizontal text interleaving across columns.	**Pass**
A scanned (image-only) PDF	Detected lack of embedded text layer; cleanly returned alert prompting for OCR PDF.	**Partial (Handled)**
A skill outside the 12 categories — tagged OTHER?	Non-canonical skills like 'Agile Scrum' and 'UI Wireframing' correctly routed to OTHER.	**Pass**
Same skill worded differently in JD and resume (e.g. 'Postgres' vs 'SQL databases') — matched?	Matched via shared RADIX category code (SQL) and token synonym dictionary.	**Pass**

8. Results on Sample Data

Candidate & Expected Fit	Talent Check: Google LLC	Skill Match: Google SWE	Skill Match: Oracle FSS	Matches Expected Fit?
Ananya Rao Expected: Strong SWE, weaker data roles	73.68 / 100 (Ready: YES)	62.0% Match (Strong algorithms)	31.0% Match (Lacks DB support)	**YES (Y)**
Karthik Subramaniam Expected: Strong Data Scientist, weaker pure SWE	31.58 / 100 (Ready: NO)	23.0% Match (High DSA deficit)	28.0% Match (Missing support)	**YES (Y)**
Priya Menon Expected: Best Oracle FSS, weaker Google/MSFT	38.50 / 100 (Ready: NO)	29.0% Match (System design deficit)	78.0% Match (Strong SQL/App support)	**YES (Y)**
Rohan Verma Expected: Competitive across nearly every role	88.24 / 100 (Ready: YES)	95.0% Match (Superior SWE depth)	85.0% Match (Broad multi-skill coverage)	**YES (Y)**

Sample Data Alignment Analysis

Explanation of Results & Stated Fit Agreement: Zero disagreements ('N') occurred. The platform exhibited clear discrimination across candidate profiles: Ananya cleared Google SWE baseline (73.68) but scored low on Oracle FSS (31%); Karthik showed expected data strengths but failed Google's pure SWE algorithms (31.58); Priya matched Oracle FSS strongly (78%) while showing gaps for Google; Rohan performed competitively across all roles (88.24 readiness, 95% SWE match).

9. How to Run Our Code

Prerequisites	Python 3.10+ (tested on Python 3.13), Node.js v18+ (tested on v22), PowerShell 5.1/7+
Setup Commands	pip install -r requirements.txt ; cd radix-talent-match_frontend && npm install --legacy-peer-deps && cd ..
Environment Variables	GROQ_API_KEY (optional, fallback parser active), SUPABASE_URL, SUPABASE_KEY (local JSON cache active)
Start Command (1-Click)	powershell -ExecutionPolicy Bypass -File .\scripts\start-all.ps1 (Runs Ports 8001–8005 & Vite 5173)
Sample Data Location	data/ (resumes, JDs, profiles.json) and Talent-match/backend/data/company_skillsets.json

10. Definition of Done

[X] All 5 pieces run on their own against at least 2 sample JDs / resumes
[X] The 5 pieces are wired into one end-to-end flow for at least one company / JD / candidate
[X] Talent Check and Skill Matching give visibly different results for different candidates
[X] Extraction accuracy measured on at least 2 JDs and 2 resumes (Section 7a: >90% Precision/Recall)
[X] Code runs from the README steps on a clean machine (Verified automated 12-test suite)

11. Challenge, Learnings, Limitations & Next Steps

One Challenge We Faced	Integrating five independent FastAPI microservices with differing schema keys and port collisions on Windows. We resolved this by centralizing shared Pydantic data schemas in common/ and creating unified PowerShell automation scripts (start-all.ps1 and stop-all.ps1) with automated port health validation.
What We Learned	Locking the data contract on Day 1 is the single highest-ROI activity in distributed engineering. Building resilient local cache fallbacks decoupled frontend testing from external API limits, preventing blockers across the entire 5-day cycle.
Known Limitations	1. Scanned image-only PDFs without text layers require OCR preprocessing. 2. Projects and hackathons currently rely on self-reported proficiencies rather than live GitHub commit verification.
Next Steps	1. Integrate automated GitHub repository analysis for automated coding verification. 2. Add dynamic AI mock interview generation customized to the candidate's exact identified skill gaps.

12. Demo Video (15 Minutes)

Video File Name: Demo_Video.mp4 in Google Drive folder (Strictly 15:00 minutes runtime)

Time	Presenter	What to Show
0:00 – 1:30	**Scrum Master (Aryan Nigam)**	Team introduction, the problem, the integrated flow at a glance, and what makes your solution different
1:30 – 4:00	**Role 1 — JD Analytics (Aman Sharma)**	Upload a JD and show the extracted skill list; mention your precision / recall
4:00 – 6:30	**Role 2 — Resume Parsing (Aryan Nigam)**	Upload a resume and show the extracted skills and details; show one messy-layout case
6:30 – 9:00	**Role 3 — Profile Builder (Khushi Goel)**	Profile pre-filled from the resume, edited and saved (proficiency sliders & cloud/cache sync)
9:00 – 11:30	**Role 4 — Talent Check (Vaibhav Maurya)**	Readiness score and per-skillset gaps for a selected company; explain how 'ready' is defined
11:30 – 14:00	**Role 5 — Skill Matching (Keshav Agrawal)**	Match score with matched and missing skills; a second candidate to show different results
14:00 – 15:00	**Scrum Master (Aryan Nigam)**	One challenge faced, how it was solved, known limitations, and next steps

13. Declarations

AI Tools Used	1. Groq Cloud API (Meta LLaMA-3.3-70B-versatile and LLaMA-3.1-8B-instant) embedded in Module 1 & Module 2 pipelines for semantic text extraction and RADIX categorization. 2. GitHub Copilot & Cursor AI used during development for syntax completion, boilerplate typing, and unit test scaffold creation.
Data Authenticity	We confirm that only the official supplied sample job descriptions, resumes, and company skillset benchmarks provided in the RADIX Talent Match Hackathon Pack were used for evaluation and benchmarking.
Originality & Authorship	We confirm that all microservice architecture, API integration pipelines, scoring algorithms, frontend components, and documentation are the original work of Team Piratech, engineered specifically for the RADIX Talent Match Hackathon.